

Práctica Control Bajo Nivel

Ejercicio 1:

Función usada:

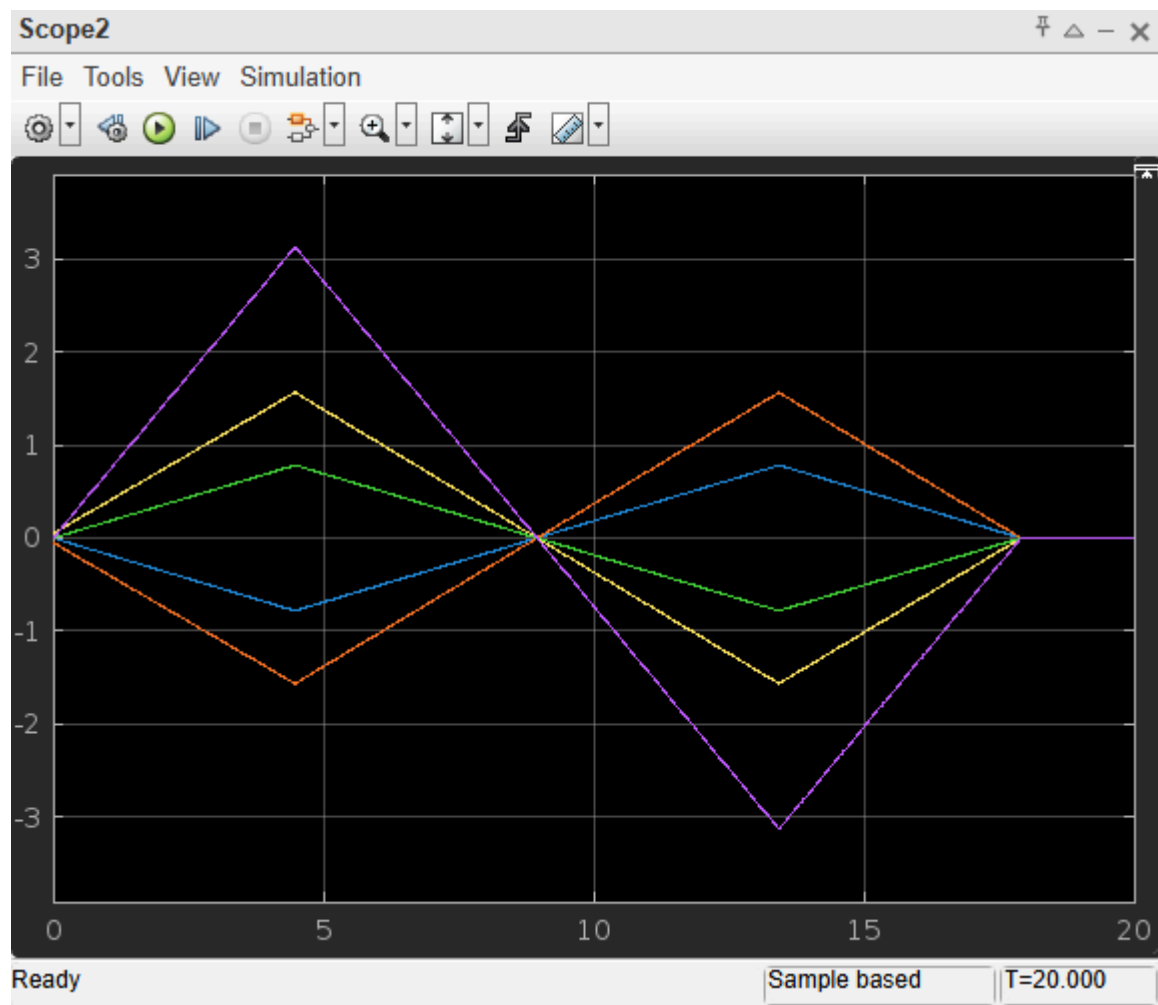
```
function [v1,v2,v3,v4,v5] = fcn(q1,q2,q3,q4,q5,v_max)
    persistent etapa;
    if isempty(etapa)
        etapa=1;
    end
    qA = [0,0,0,0,0];
    qB = [pi/2, -pi/4, -pi/2, pi/4, pi];
    qC = [-pi/2, pi/4, pi/2, -pi/4, -pi];
    switch etapa
        case 1
            objetivo = qB;
        case 2
            objetivo = qA;
        case 3
            objetivo = qC;
        case 4
            objetivo = qA;
        otherwise
            objetivo = qA;
    end
    actual = [q1,q2,q3,q4,q5];
    distancia = objetivo - actual;
    max_dist = max(abs(distancia));
    if max_dist < 0.01 && etapa < 5
        etapa = etapa + 1;
    end
    vel_factor = distancia / max_dist;
    vel = distancia .* vel_factor * 1000;
    for i = 1:5
        if vel(i) > v_max
            vel(i) = v_max * vel_factor(i);
        elseif vel(i) < -v_max
            vel(i) = -v_max * vel_factor(i);
        end
    end
    v1 = vel(1);
    v2 = vel(2);
    v3 = vel(3);
    v4 = vel(4);
    v5 = vel(5);
end
```

Explicación:

En esta función se hace que el robot siga una secuencia de posiciones: $A \rightarrow B \rightarrow A \rightarrow C \rightarrow A$. Para ello, se calcula la distancia entre la posición actual y la deseada, y se generan velocidades para cada articulación de forma que todas lleguen al mismo tiempo, sin superar una velocidad máxima. Cuando se alcanza una posición, la función pasa automáticamente a la siguiente.

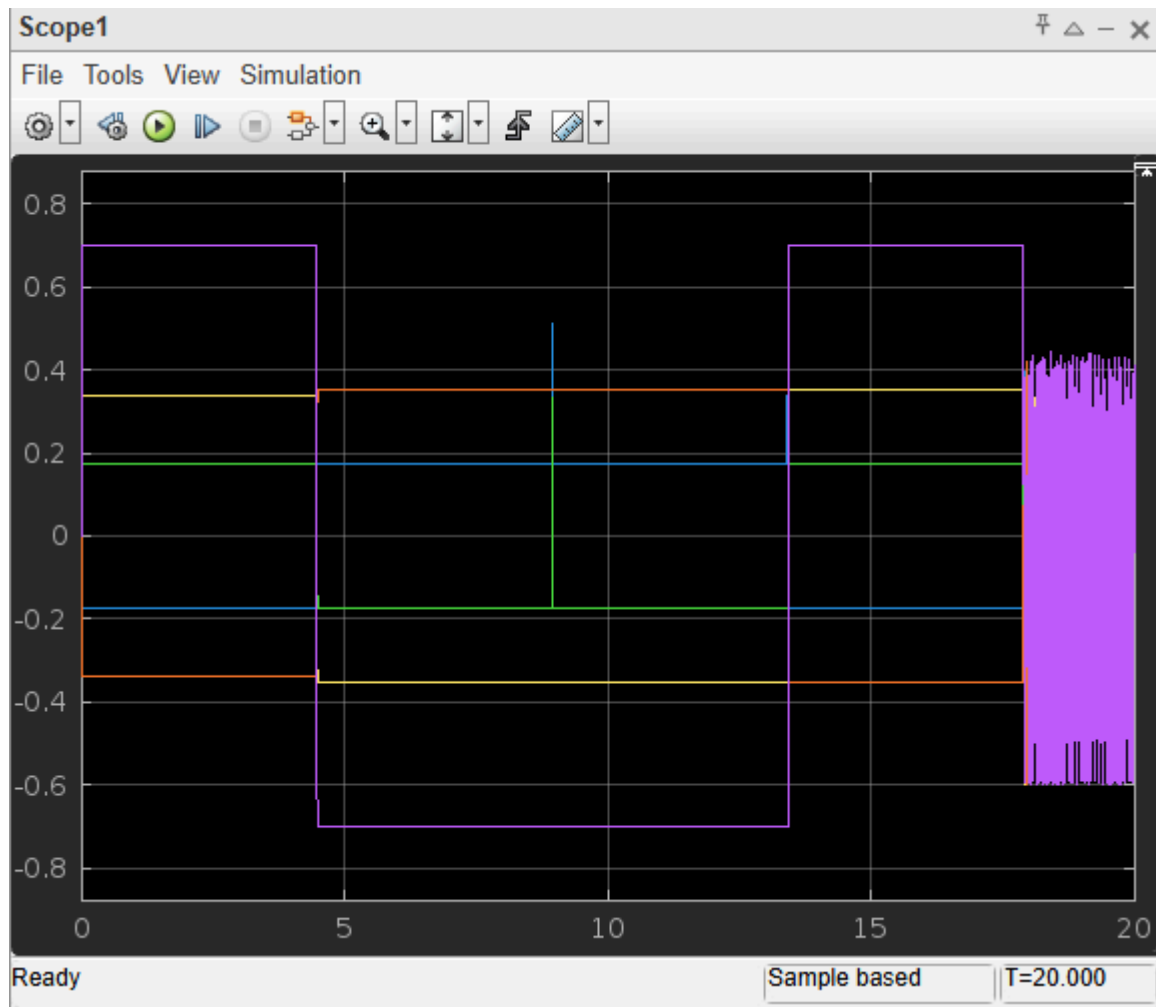
Gráficas de los Scopes:

Posiciones articulares



En la gráfica se muestran las posiciones de las cinco articulaciones del robot durante la secuencia $A \rightarrow B \rightarrow A \rightarrow C \rightarrow A$. Se observa cómo todas las articulaciones alcanzan cada punto de forma coordinada y sincronizada, volviendo a la posición inicial entre cada movimiento.

Velocidades



En esta gráfica se representan las velocidades de cada articulación durante el movimiento entre los puntos A, B y C. Se observa cómo las velocidades se mantienen constantes en cada tramo, ajustándose para que todas las articulaciones lleguen al mismo tiempo. Al cambiar de objetivo, las velocidades cambian bruscamente para cambiar de objetivo.

Ejercicio 2:

Dibujo del robot:

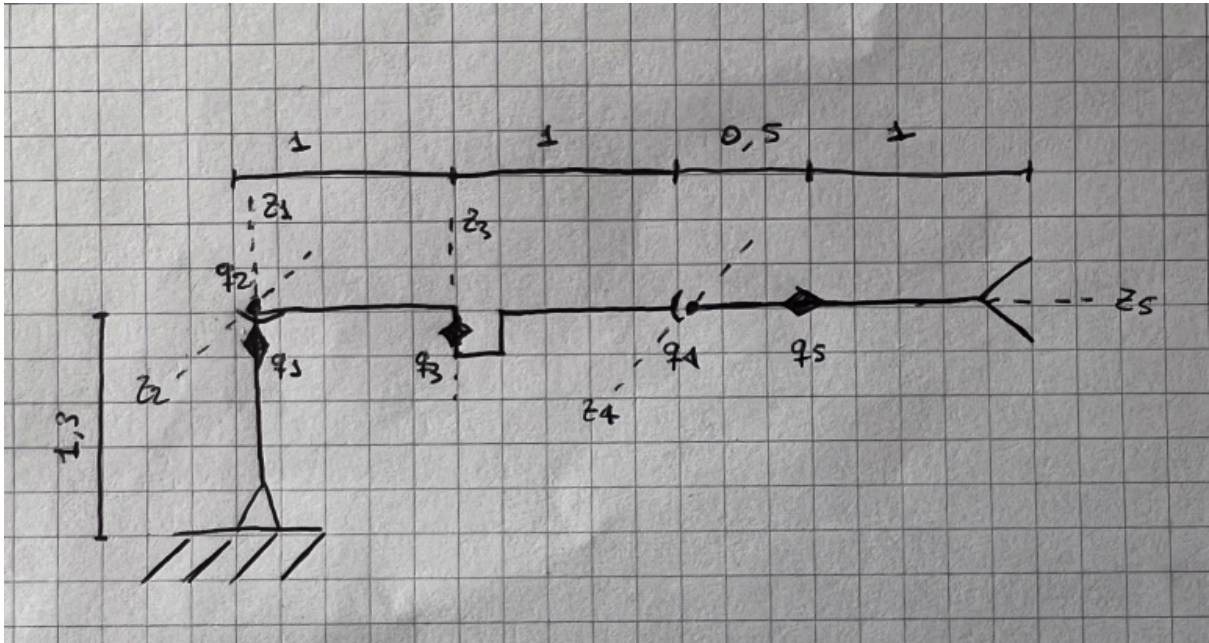


Tabla de Denavit-Hatemberg:

i	θ	d	a	α
1	q_1	l_1	0	+90
2	q_2	0	l_2	-90
3	q_3	0	l_3	+90
4	q_4	0	l_4+l_5	0
5	q_5	0	0	+90

Cinemática directa

```
T = simplify(A0_1 * A1_2 * A2_3 * A3_4 * A4_5)
```

T =

$$\begin{pmatrix} -\cos(q_1+q_3)\sin(q_4+q_5) & \sin(q_1+q_3) & \cos(q_1+q_3)\cos(q_4+q_5) & l_3\cos(q_1+q_3)+l_2\cos(q_1)+\cos(q_1+q_3)\cos(q_4)(l_4+l_5) \\ -\sin(q_1+q_3)\sin(q_4+q_5) & -\cos(q_1+q_3) & \cos(q_4+q_5)\sin(q_1+q_3) & l_3\sin(q_1+q_3)+l_2\sin(q_1)+\sin(q_1+q_3)\cos(q_4)(l_4+l_5) \\ \cos(q_4+q_5) & 0 & \sin(q_4+q_5) & l_1+\sin(q_4)(l_4+l_5) \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

```
Px = T(1, 4)
```

$$Px = l_3\cos(q_1+q_3)+l_2\cos(q_1)+\cos(q_1+q_3)\cos(q_4)(l_4+l_5)$$

```
Py = T(2, 4)
```

$$Py = l_3\sin(q_1+q_3)+l_2\sin(q_1)+\sin(q_1+q_3)\cos(q_4)(l_4+l_5)$$

```
Pz = T(3, 4)
```

$$Pz = l_1+\sin(q_4)(l_4+l_5)$$

Cinemática Inversa

Sabemos que $q_2=0$ y $q_5=0$, pero no es necesario sustituir en P_x, P_y, P_z ya que no dependen de ellos

```
syms X Y Z
```

```
% Resolvemos q1 y q3 a partir de X, Y
```

```
Solution = solve([Px==X, Py==Y], [q1,q3], 'Real', true);
```

Warning: Solutions are only valid under certain conditions. To include parameters and conditions in the solution, specify the 'ReturnConditions' value as 'true'.

```
solq1 = simplify(Solution.q1, 'Steps', 30)
```

```
solq1 =
```

$$\begin{pmatrix} 2 \operatorname{atan}\left(\frac{\left(\sigma_1 - \frac{8 Y l_2^2 \alpha_4}{\sigma_3 \sigma_2}\right) \sigma_2}{4 l_2 \alpha_4}\right) \\ -2 \operatorname{atan}\left(\frac{\left(\sigma_1 + \frac{8 Y l_2^2 \alpha_4}{\sigma_3 \sigma_2}\right) \sigma_2}{4 l_2 \alpha_4}\right) \end{pmatrix}$$

where

$$\sigma_1 = \frac{4 l_2 \alpha_4 \sqrt{\frac{\sigma_2 (-X^2 - Y^2 + l_2^2 + 2 l_2 l_3 + 2 l_2 l_4 \cos(q_4) + 2 l_2 l_5 \cos(q_4) + l_3^2 + 2 l_3 l_4 \cos(q_4) + 2 l_3 l_5 \cos(q_4) + \sigma_7 + \sigma_5 + \sigma_6)}{\sigma_3^4}}}{\sigma_2}$$

$$\sigma_2 = -X^2 - Y^2 + l_2^2 - 2 l_2 l_3 - 2 l_2 l_4 \cos(q_4) - 2 l_2 l_5 \cos(q_4) + l_3^2 + 2 l_3 l_4 \cos(q_4) + 2 l_3 l_5 \cos(q_4) + \sigma_7 + \sigma_5 + \sigma_6$$

$$\sigma_3 = -X^2 - 2 X l_2 - Y^2 - l_2^2 + l_3^2 + 2 l_3 l_4 \cos(q_4) + 2 l_3 l_5 \cos(q_4) + \sigma_7 + \sigma_5 + \sigma_6$$

$$\alpha_4 = l_3 + l_4 \cos(q_4) + l_5 \cos(q_4)$$

$$\sigma_5 = 2 l_4 l_5 \cos(q_4)^2$$

$$\sigma_6 = l_5^2 \cos(q_4)^2$$

$$\sigma_7 = l_4^2 \cos(q_4)^2$$

```
solq3 = simplify(Solution.q3, 'Steps', 30)
```

```
solq3 =
```

$$\begin{pmatrix} -\sigma_1 \\ \sigma_1 \end{pmatrix}$$

where

$$\sigma_1 = 2 \operatorname{atan}\left(\sqrt{\frac{\sigma_2(-X^2 - Y^2 + l_2^2 + 2l_2l_3 + 2l_2l_4\cos(q_4) + 2l_2l_5\cos(q_4) + l_3^2 + 2l_3l_4\cos(q_4) + 2l_3l_5\cos(q_4) + \alpha_6 + \alpha_4 + \sigma_5)}{\sigma_2^4}}\right) \sigma_2$$

$$\sigma_2 = -X^2 - Y^2 + l_2^2 - 2l_2l_3 - 2l_2l_4\cos(q_4) - 2l_2l_5\cos(q_4) + l_3^2 + 2l_3l_4\cos(q_4) + 2l_3l_5\cos(q_4) + \alpha_6 + \alpha_4 + \sigma_5$$

$$\sigma_3 = -X^2 - 2Xl_2 - Y^2 - l_2^2 + l_3^2 + 2l_3l_4\cos(q_4) + 2l_3l_5\cos(q_4) + \alpha_6 + \alpha_4 + \sigma_5$$

$$\alpha_4 = 2l_4l_5\cos(q_4)^2$$

$$\sigma_5 = l_5^2\cos(q_4)^2$$

$$\alpha_6 = l_4^2\cos(q_4)^2$$

```
% Luego resolvemos q4 desde Z
```

```
solq4 = simplify(solve(Pz==Z, q4))
```

```
solq4 =
```

$$\begin{pmatrix} \arcsin\left(\frac{Z - l_1}{l_4 + l_5}\right) \\ \pi - \arcsin\left(\frac{Z - l_1}{l_4 + l_5}\right) \end{pmatrix}$$

Función para determinar la posición:

```
[X, Y, Z] = deal(-0.2, 2.4, 2.4);
```

```
[q1, q2, q3, q4, q5] = calcPos(X, Y, Z)
```

```
q1 = 2×1
    0.6833
    2.6246
```

```
q2 =
    0
```

```
q3 = 2×1
    1.3915
   -1.3915
```

```
q4 =
    0.8232
```

```
q5 =
    0
```

La función calcPos(X, Y, Z) recibe como parámetro las coordenadas y devuelve las q. Usa las ecuaciones calculadas anteriormente en la cinemática inversa

Ejercicio 3:

Función usada:

```
function [v1,v2,v3,v4,v5] = fcn(q1,q2,q3,q4,q5,vMax)
persistent counter objetivo1 objetivo2 objetivo3

l_1 = 1.3;
l_2 = 1;
l_3 = 1;
l_4 = 0.5;
l_5 = 1;

if isempty(counter)
    counter = 1;
end

% =====
% P1
if isempty(objetivo1)
    Px = -0.2; Py = 2.4; Pz = 2.4;
    obj_q4=asin((Pz-l_1)/(l_4+l_5));

    K=l_3+(l_4+l_5)*cos(obj_q4);
    cos_q3 = (Px^2 + Py^2 - l_2^2 - K^2) / (2 * l_2 * K);
    sin_q3 = sqrt(1 - cos_q3^2);

    q3_calc = -atan2(sin_q3, cos_q3);
    q1_calc = atan2(-Px, Py) + atan2(sqrt(Py^2 + Px^2 - (K*sin(q3_calc))^2),
K*sin(q3_calc));

    objetivo1 = [q1_calc,0,q3_calc,obj_q4,0];
end

% =====
% P2
if isempty(objetivo2)
    Px = 2.8; Py = -1; Pz = 1.8;
    obj_q4=asin((Pz-l_1)/(l_4+l_5));

    K=l_3+(l_4+l_5)*cos(obj_q4);
    cos_q3 = (Px^2 + Py^2 - l_2^2 - K^2) / (2 * l_2 * K);
    sin_q3 = sqrt(1 - cos_q3^2);

    q3_calc = -atan2(sin_q3, cos_q3);
    q1_calc = atan2(-Px, Py) + atan2(sqrt(Py^2 + Px^2 - (K*sin(q3_calc))^2),
K*sin(q3_calc));

    objetivo2 = [q1_calc,0,q3_calc,obj_q4,0];
end

% =====
% P3
if isempty(objetivo3)
    Px = 1.4; Py = -1.4; Pz = 0;
    obj_q4=asin((Pz-l_1)/(l_4+l_5));
```



```

K=l_3+(l_4+l_5)*cos(obj_q4);
cos_q3 = (Px^2 + Py^2 - l_2^2 - K^2) / (2 * l_2 * K);
sin_q3 = sqrt(1 - cos_q3^2);

q3_calc = -atan2(sin_q3, cos_q3);
q1_calc = atan2(-Px, Py) + atan2(sqrt(Py^2 + Px^2 - (K*sin(q3_calc))^2),
K*sin(q3_calc));

objetivo3 = [q1_calc,0,q3_calc,obj_q4,0];
end

% =====
% SECUENCIA
if counter > 3
    counter = 3;
end

objetivos = {objetivo1, objetivo2, objetivo3};
actual = [q1,q2,q3,q4,q5];
objetivo = objetivos{counter};

distancia = objetivo - actual;
max_dist = max(abs(distancia));

if max_dist < 0.01 && counter < 3
    counter = counter + 1;
end

vel_factor = abs(distancia / max_dist);
vel = distancia .* vel_factor * 1000;

for i = 1:5
    if vel(i) > vMax
        vel(i) = vMax * vel_factor(i);
    elseif vel(i) < -vMax
        vel(i) = -vMax * vel_factor(i);
    end
end

v1 = vel(1); v2 = vel(2); v3 = vel(3); v4 = vel(4); v5 = vel(5);
end

```

Explicación:

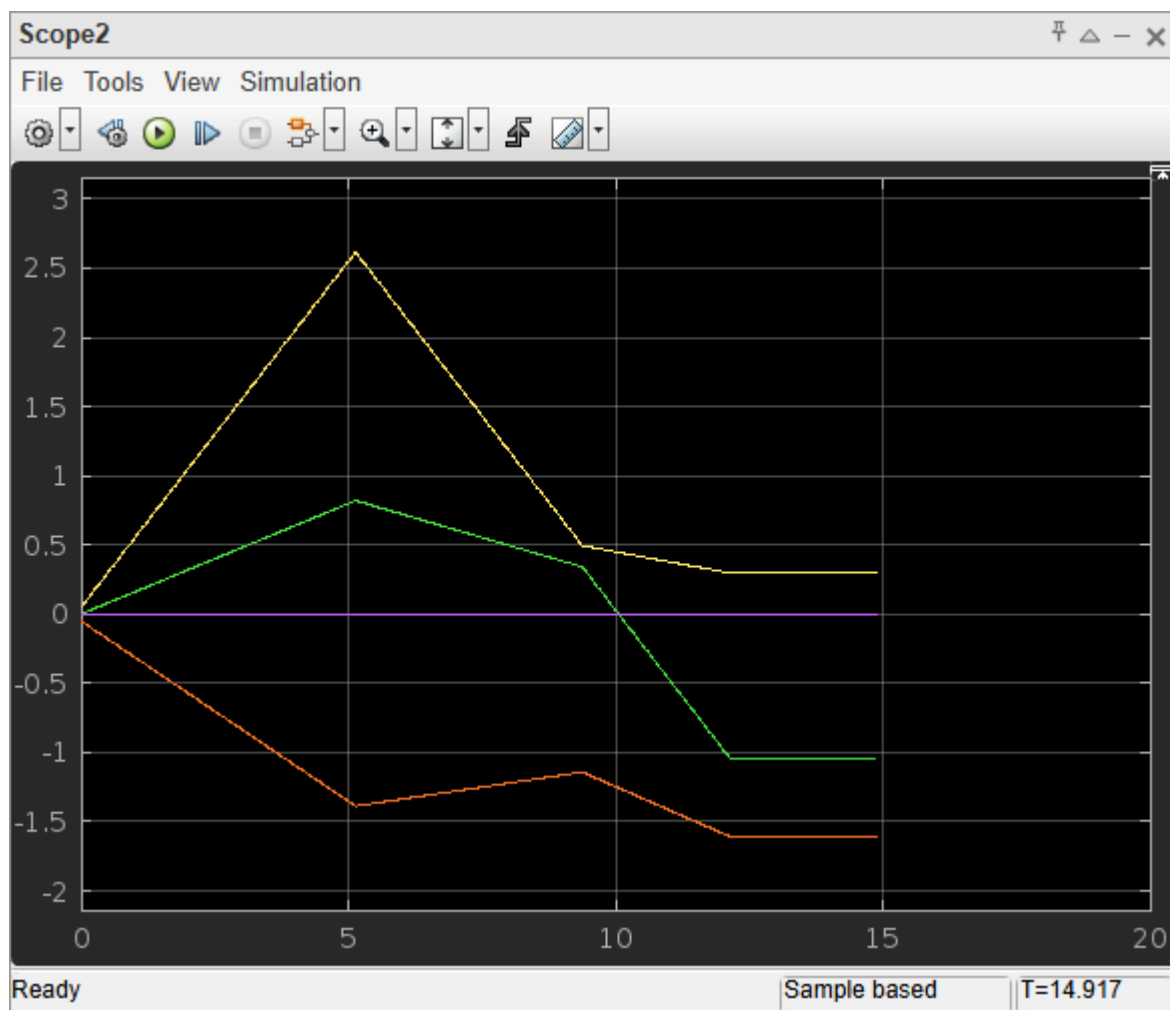
En este ejercicio se realiza un control del robot que le permite pasar por tres puntos definidos: P1, P2 y P3. A diferencia del ejercicio 1, los puntos no están dados en coordenadas articulares, por lo que es necesario transformar las posiciones (Px, Py, Pz) a coordenadas articulares (q1, q3 y q4).

Primero, se calcula q4 mediante la función “asin”, utilizando la altura del punto (Pz) y las longitudes de los eslabones finales del robot. Luego, se obtiene q3 usando el teorema del coseno, a partir del plano horizontal y la geometría de los eslabones l2, l3, l4 y l5. Finalmente, q1 se calcula con atan2.

Una vez obtenidos los ángulos, se genera una secuencia de objetivos articulares que el robot sigue sin detenerse. La velocidad de cada articulación se ajusta dinámicamente, garantizando que todas terminen su movimiento al mismo tiempo y sin superar una velocidad máxima (v_{Max}).

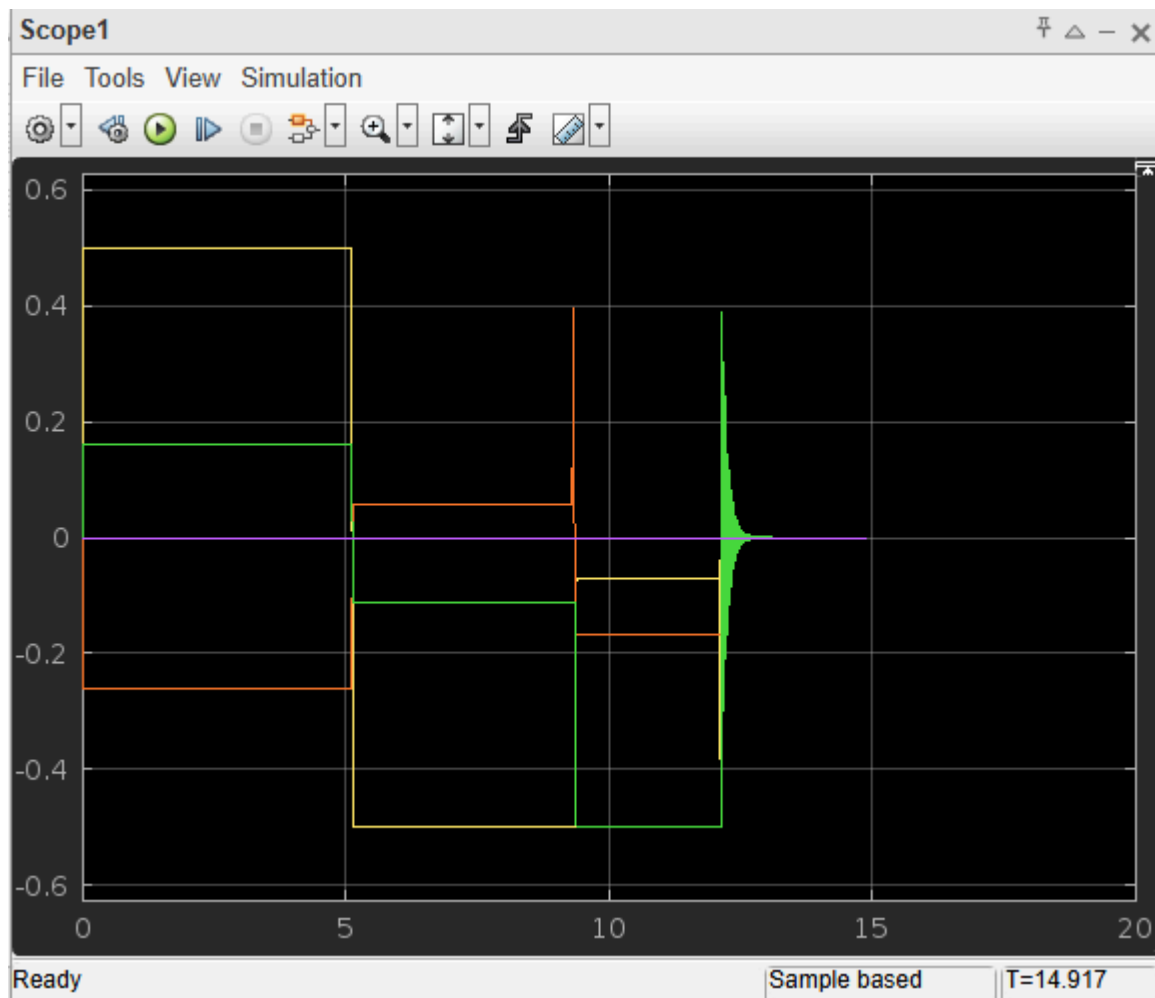
Gráficas de los Scopes:

Posición:



En esta gráfica, se observa cómo las articulaciones del robot cambian de posición de forma coordinada a medida que se alcanzan los tres puntos objetivos P1, P2 y P3. Las curvas muestran transiciones suaves entre los valores de cada articulación. Se aprecia claramente que las posiciones se estabilizan entre cada tramo, lo que indica que el robot alcanza cada destino antes de avanzar al siguiente. Se ve como el movimiento se realiza sin detenerse.

Velocidad:



Esta gráfica representa las velocidades de las articulaciones durante el movimiento. Se puede ver cómo las velocidades se mantienen constantes en cada tramo del movimiento, adaptándose según la articulación que recorra mayor distancia. Las transiciones entre velocidades reflejan el paso entre puntos, y se puede observar cómo, al acercarse al objetivo, las velocidades disminuyen hasta estabilizarse, especialmente en los últimos instantes del trayecto. Pero nunca llegando a detenerse.

Ejercicio 4:

Funciones usadas:

```
function [v1,v2,v3,v4,v5] = fcn( q1,q2,q3,q4,q5,vMax)
persistent counter counterStart starter finish objCurrent line;

l1 = 1.3; l2 = 1; l3 = 1; l4 = 0.5; l5 = 1;

num_puntos = 1000;

%Definimos las variables persistentes y los puntos
if isempty(counter)
    counter = 0;
end

if isempty(counterStart)
    counterStart = 0;
end

if isempty(line)
    P4 = [3.25,0,1.3];
    P5 = [0.5,-2.25,0.1];
    line = (P5-P4)/num_puntos;
end

% Definimos la configuracion articular correspondiente al punto inicial
if isempty(starter)
    Px = 3.25;
    Py = 0;
    Pz = 1.3;
    obj_q4=asin((Pz-l1)/(l4+l5));

    K=l3+(l4+l5)*cos(obj_q4);
    obj_q3 = [acos((Px^2+Py^2-l2^2-K^2)/(2*l2*K)),
    -acos((Px^2+Py^2-l2^2-K^2)/(2*l2*K))];

    obj_q1=atan2(-Px,Py)+atan2(sqrt(Py.^2+(-Px).^2-(K*sin(obj_q3)).^2),
    K*sin(obj_q3));
    starter = [obj_q1(1,2),0,obj_q3(1,2),obj_q4,0];
end

% Definimos la configuracion articular correspondiente al punto final
if isempty(finish)
    Px = 0.5;
    Py = -2.25;
    Pz = 0.1;
    obj_q4=asin((Pz-l1)/(l4+l5));

    K=l3+(l4+l5)*cos(obj_q4);
    obj_q3 = [acos((Px^2+Py^2-l2^2-K^2)/(2*l2*K)),
    -acos((Px^2+Py^2-l2^2-K^2)/(2*l2*K))];

    obj_q1=atan2(-Px,Py)+atan2(sqrt(Py.^2+(-Px).^2-(K*sin(obj_q3)).^2),
    K*sin(obj_q3));
    finish = [obj_q1(1,2),0,obj_q3(1,2),obj_q4,0];
end
```

```

if isempty(objCurrent)
    objCurrent = starter;
end

if counter > num_puntos
    counter = num_puntos;
end

position = [q1,q2,q3,q4,q5];
distance = objCurrent - position;
max_dist = max(abs(distance));

% Vemos si hemos alcanzado el objetivo actual
if (max_dist) < 0.001
    % Comprobamos si hemos empezado a movernos
    if counter == 0
        % Esperamos un tiempo inicial antes de movernos
        if counterStart > num_puntos
            counter = 1;
        end
        counterStart = counterStart + 1;
    else
        counter = counter + 1;
    end
    %Calculamos la nueva posicion sobre la linea recta
    Px = 3.25 + line(1,1)*counter;
    Py = line(1,2)*counter;
    Pz = 1.3 + line(1,3)*counter;

    % Calculamos q4 con la proyeccion de Z
    obj_q4=asin((Pz-11)/(14+15));
    % Calculamos la longitud efectiva desde el eje hasta la mano
    K=13+(14+15)*cos(obj_q4);
    % Usamos la ley del coseno para encontrar q3
    obj_q3 = [acos((Px^2+Py^2-12^2-K^2)/(2*12*K)),
    -acos((Px^2+Py^2-12^2-K^2)/(2*12*K))];
    % Calculamos q1 con la arcotangente
    obj_q1=atan2(-Px,Py)+atan2(sqrt(Py.^2+(-Px).^2-(K*sin(obj_q3)).^2),
    K*sin(obj_q3));
    objCurrent = [obj_q1(1,2),0,obj_q3(1,2),obj_q4,0];

end

% Calculamos la velocidad correspondiente a cada articulacion
velocidades = abs(distance / max_dist);

v1 = distance(1,1)*velocidades(1,1)*num_puntos;
if v1 > vMax
    v1 = vMax * velocidades(1,1);
elseif v1 < -vMax
    v1 = -vMax * velocidades(1,1);
end

v3 = distance(1,3)*velocidades(1,3)*num_puntos;
if v3 > vMax
    v3 = vMax * velocidades(1,3);
elseif v3 < -vMax
    v3 = -vMax * velocidades(1,3);
end

```

```

v4 = distance(1,4)*velocidades(1,4)*num_puntos;
if v4 > vMax
    v4 = vMax * velocidades(1,4);
elseif v4 < -vMax
    v4 = -vMax * velocidades(1,4);
end

v2 = 0;
v5 = 0;

```

Explicación:

Para la resolución de este ejercicio es necesario establecer control en posición.

Hemos incluido una función que calcula las posiciones x, y, z, que es la denominada fcn2:

```

function [x,y,z] = fcn2(q1,q2,q3,q4,q5)

l1 = 1.3; l2 = 1; l3 = 1; l4 = 0.5; l5 = 1;

% Calculamos la matriz homogénea que calcula de transformacion de la base
% al end-effector
T = [cos(q5)*(sin(q4)*(sin(q1)*sin(q3) - cos(q1)*cos(q2)*cos(q3)) -
cos(q1)*cos(q4)*sin(q2)) + sin(q5)*(cos(q3)*sin(q1) + cos(q1)*cos(q2)*sin(q3)),
cos(q5)*(cos(q3)*sin(q1) + cos(q1)*cos(q2)*sin(q3)) -
sin(q5)*(sin(q4)*(sin(q1)*sin(q3) - cos(q1)*cos(q2)*cos(q3)) -
cos(q1)*cos(q4)*sin(q2)), - cos(q4)*(sin(q1)*sin(q3) - cos(q1)*cos(q2)*cos(q3)) -
cos(q1)*sin(q2)*sin(q4), l2*cos(q1)*cos(q2) - l3*sin(q1)*sin(q3) - cos(q4)*(l4 +
l5)*(sin(q1)*sin(q3) - cos(q1)*cos(q2)*cos(q3)) - cos(q1)*sin(q2)*sin(q4)*(l4 + l5)
+ l3*cos(q1)*cos(q2)*cos(q3); - cos(q5)*(sin(q4)*(cos(q1)*sin(q3) +
cos(q2)*cos(q3)*sin(q1)) + cos(q4)*sin(q1)*sin(q2)) - sin(q5)*(cos(q1)*cos(q3) -
cos(q2)*sin(q1)*sin(q3)), sin(q5)*(sin(q4)*(cos(q1)*sin(q3) +
cos(q2)*cos(q3)*sin(q1)) + cos(q4)*sin(q1)*sin(q2)) - cos(q5)*(cos(q1)*cos(q3) -
cos(q2)*sin(q1)*sin(q3)), cos(q4)*(cos(q1)*sin(q3) + cos(q2)*cos(q3)*sin(q1)) -
sin(q1)*sin(q2)*sin(q4), l2*cos(q2)*sin(q1) + l3*cos(q1)*sin(q3) + cos(q4)*(l4 +
l5)*(cos(q1)*sin(q3) + cos(q2)*cos(q3)*sin(q1)) - sin(q1)*sin(q2)*sin(q4)*(l4 + l5)
+ l3*cos(q2)*cos(q3)*sin(q1); cos(q5)*(cos(q2)*cos(q4) - cos(q3)*sin(q2)*sin(q4)) +
sin(q2)*sin(q3)*sin(q5), cos(q5)*sin(q2)*sin(q3) - sin(q5)*(cos(q2)*cos(q4) -
cos(q3)*sin(q2)*sin(q4)), cos(q2)*sin(q4) + cos(q3)*cos(q4)*sin(q2), l1 +
l2*sin(q2) + cos(q2)*sin(q4)*(l4 + l5) + l3*cos(q3)*sin(q2) +
cos(q3)*cos(q4)*sin(q2)*(l4 + l5); 0, 0, 0, 1];

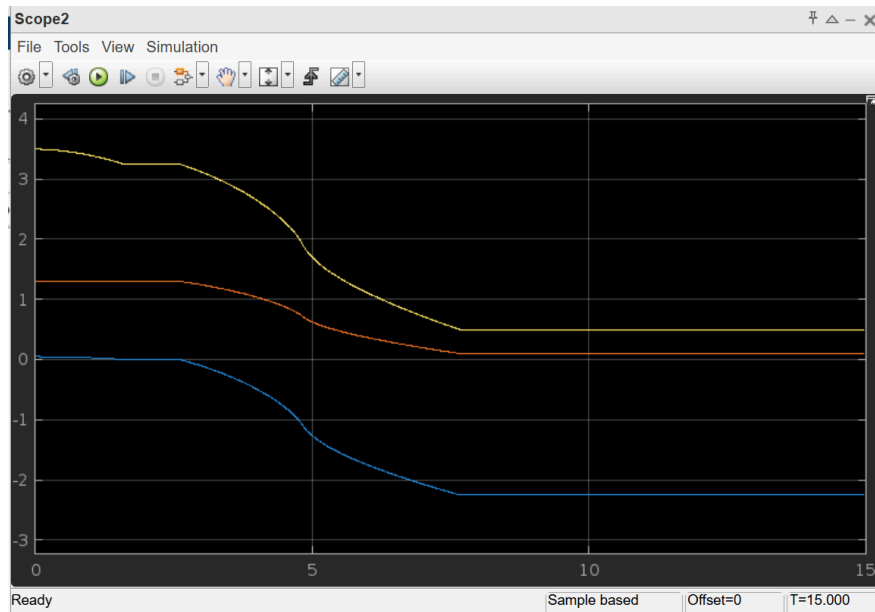
x = T(1,4);
y = T(2,4);
z = T(3,4);

```

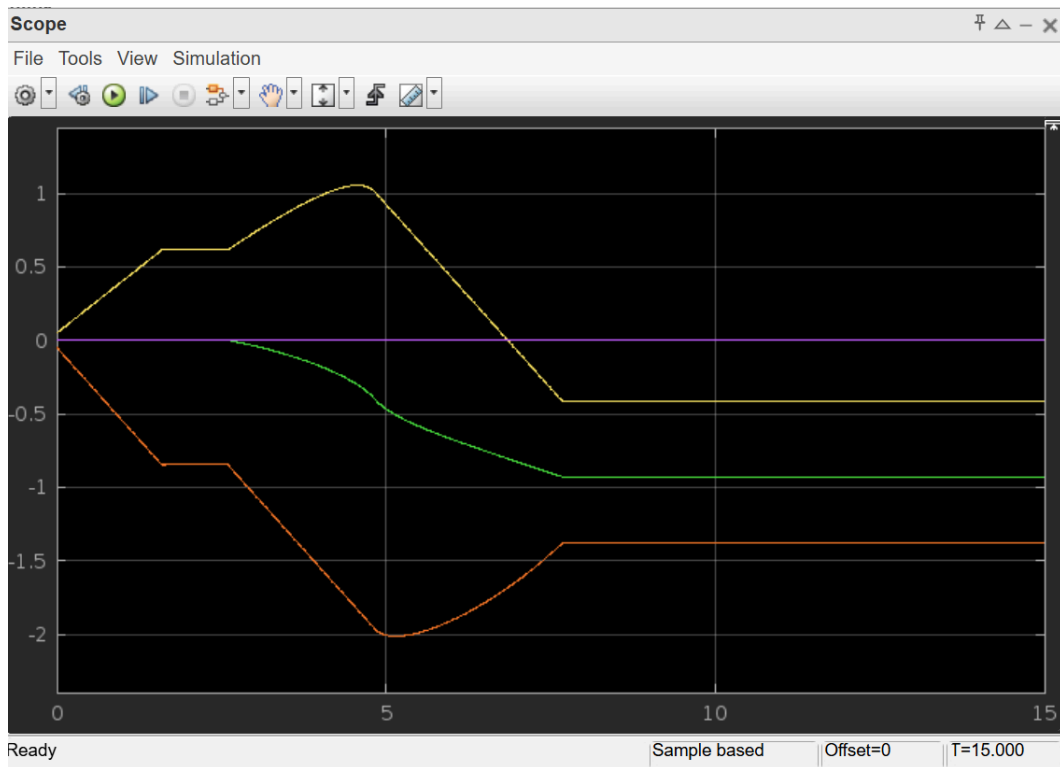
En la función fcn, calculamos la recta con los puntos pedidos. La cantidad de puntos de la recta depende de la variable num_puntos, cuanto más grande, más puntos la forman, por lo que también va más despacio..

Cabe destacar que v2 y v5 no son importantes ya que q2 y q5 son siempre 0, por lo que los definimos a 0 permanentemente.

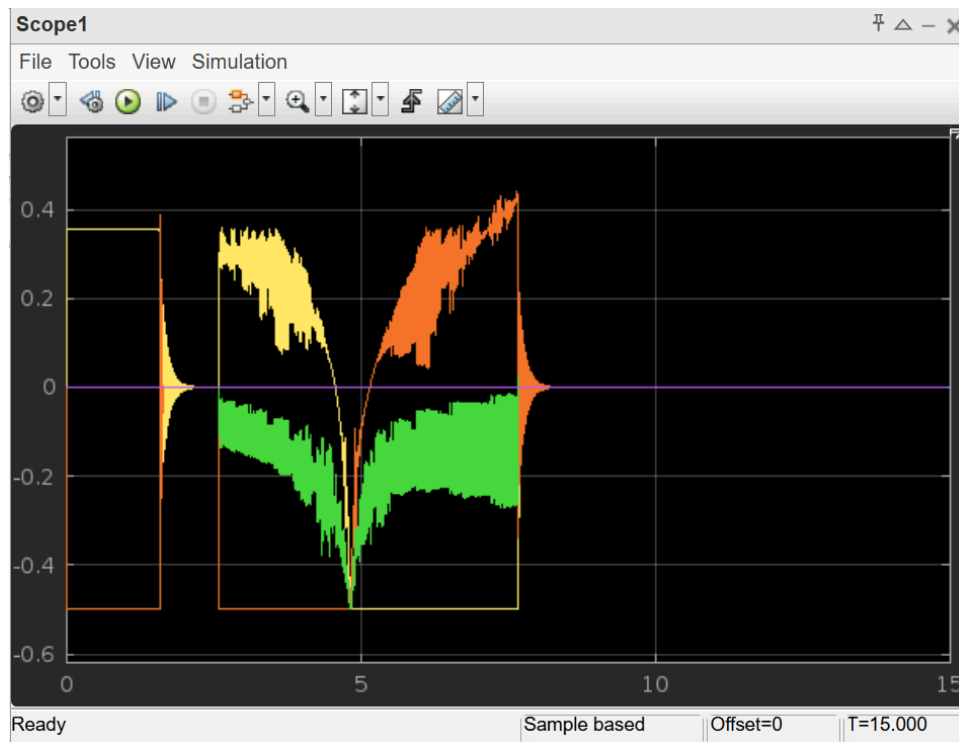
En el video vemos como el brazo primero va a la posición inicial, y luego hace una línea recta.



En la imagen vemos como en el punto intermedio baja un poco. Si bajamos la resolución, baja más rápidamente y viceversa. Al final está constante porque el brazo se queda parado.



En esta otra imagen vemos las posiciones.



En esta otra gráfica vemos las velocidades, que nunca superan 0.5, que es la velocidad que hemos preestablecido..

Ejercicio 5:

Apartado 1:

```
% Declaración simbólica de variables articulares y longitudes
syms t1 t3 t4 L1 L2 L3 L4 L5

% Construcción alternativa de la posición del extremo (end-effector)
x_pos = cos(t1 + t3)*cos(t4)*(L4 + L5) + L2*cos(t1) + L3*cos(t1 + t3)
y_pos = L2*sin(t1) + sin(t1 + t3)*cos(t4)*(L4 + L5) + L3*sin(t1 + t3)
z_pos = sin(t4)*(L4 + L5) + L1

% Agrupamos la posición en un vector
P = [x_pos; y_pos; z_pos]

% Derivamos la Jacobiana respecto a las variables articulares activas
Q = [t1; t3; t4]

J = jacobian(P, Q)

% Determinante para análisis de singularidades
J_det = det(J)

% Resolución simbólica del sistema: det(J) = 0
singulares = solve(J_det == 0, [t1,t3,t4])

% Simplificamos cada una de las soluciones simbólicas
sol_q1 = simplify(singulares.t1)
sol_q3 = simplify(singulares.t3)
sol_q4 = simplify(singulares.t4)

% Inversa simbólica de la Jacobiana
J_inv = simplify(inv(J))
```

Explicación:

Se calcula la cinemática diferencial del robot, que permite relacionar las velocidades de las articulaciones con el movimiento del extremo (TCP). Para ello, se define la posición del TCP en función de las variables articulares activas (q_1 , q_3 y q_4) usando trigonometría, y luego se deriva esta posición respecto a esas articulaciones para obtener la matriz Jacobiana. Esta matriz indica cómo cambian las coordenadas espaciales del TCP cuando cambian las articulaciones. Finalmente, se calcula el determinante de la Jacobiana y se resuelve cuándo

es cero para identificar configuraciones singulares, es decir, posiciones del robot donde pierde grados de libertad o se vuelve menos preciso.

Apartado 2:

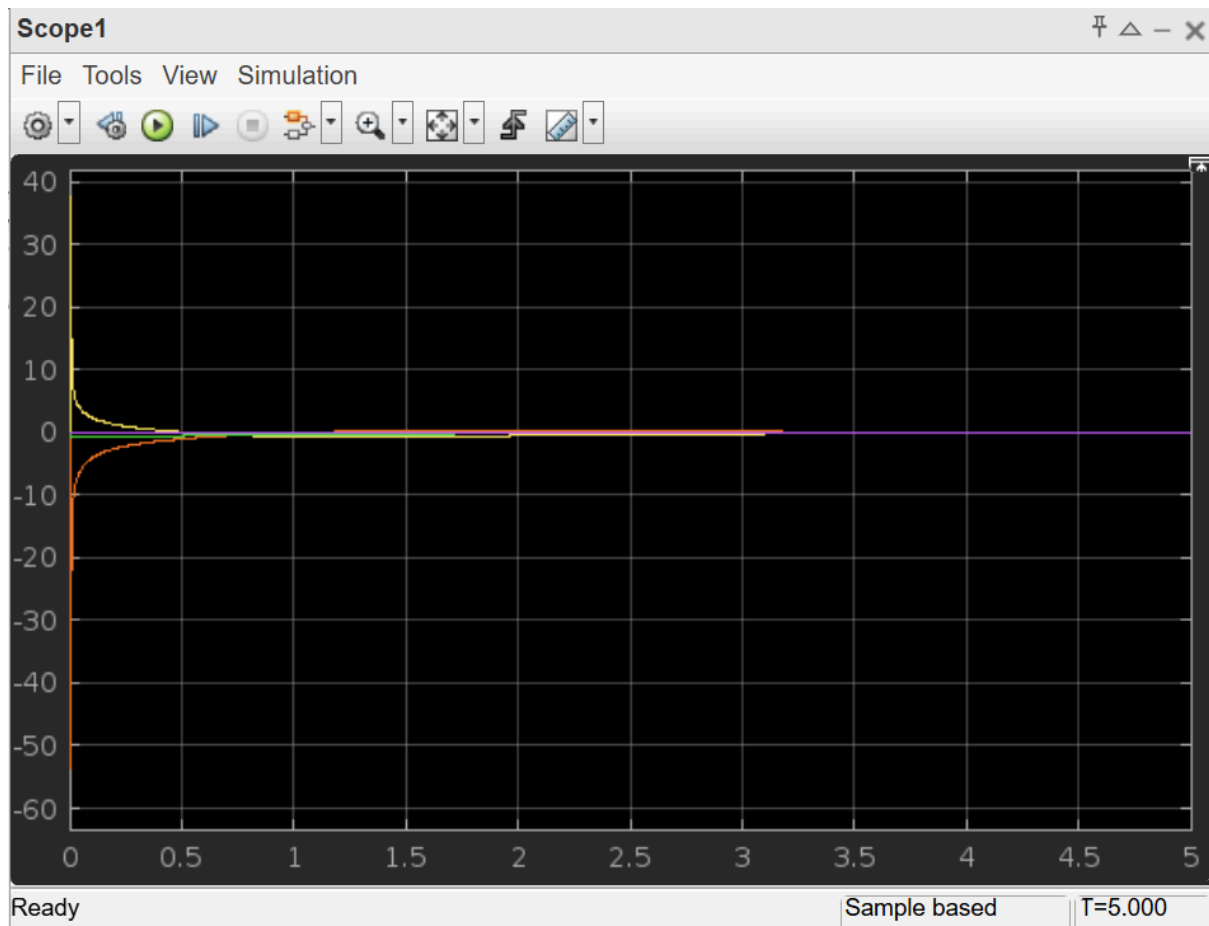
En la función `fcn`, comenzamos calculando la jacobiana inversa del sistema de forma analítica. Lo hacemos porque necesitamos transformar un desplazamiento cartesiano en el espacio (la diferencia entre la posición actual del extremo del robot y el objetivo deseado) en velocidades articulares. Esta jacobiana está simplificada, ya que solo estamos controlando las articulaciones q_1 , q_3 y q_4 .

A continuación, obtenemos la posición actual del TCP utilizando una matriz de transformación homogénea previamente desarrollada. De esa matriz extraemos las coordenadas x , y y z del extremo del robot.

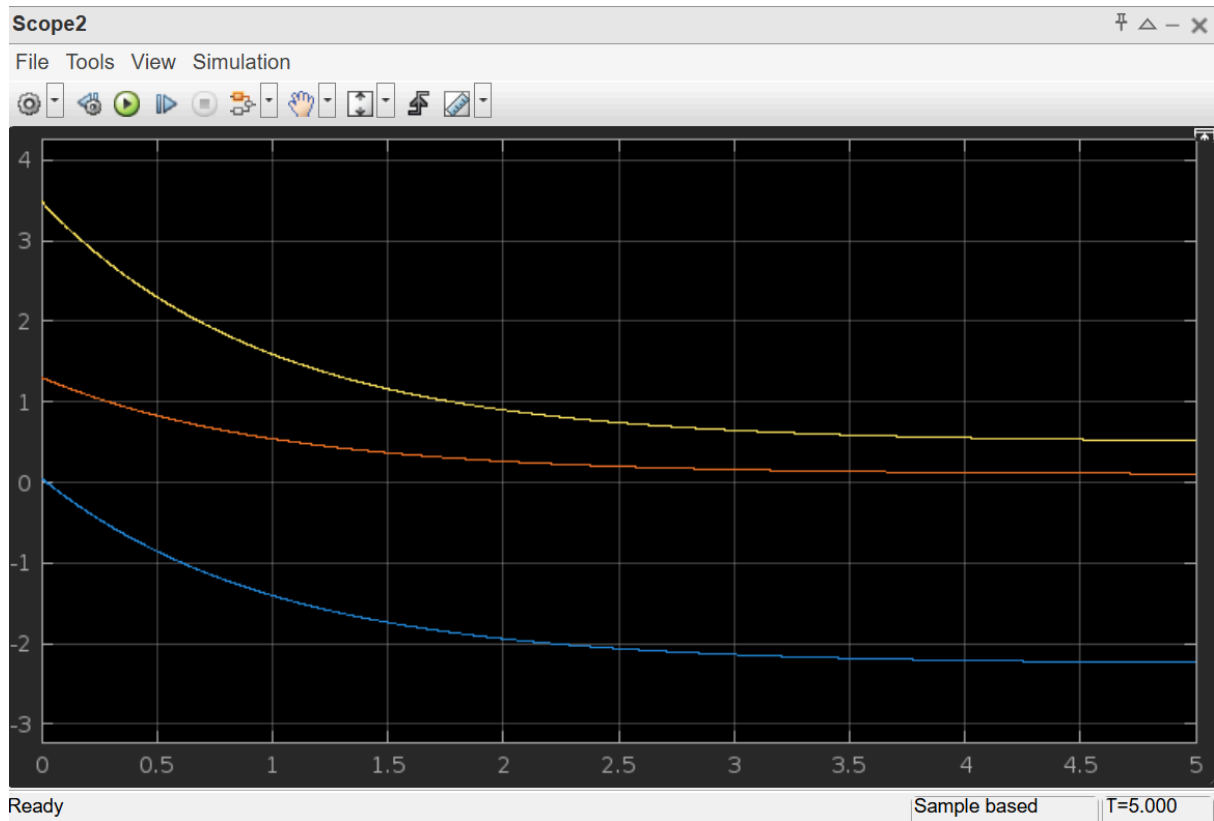
Después, calculamos el vector de error cartesiano restando la posición actual del TCP a la posición objetivo P_5 . Ese vector indica hacia dónde debe moverse el robot en el espacio.

Multiplicamos este vector de error por la jacobiana inversa para obtener las velocidades articulares necesarias para que el robot corrija su posición. Finalmente, asignamos estas velocidades solo a q_1 , q_3 y q_4 , dejando fijas q_2 y q_5 , ya que en este ejercicio no están implicadas en el movimiento del TCP.

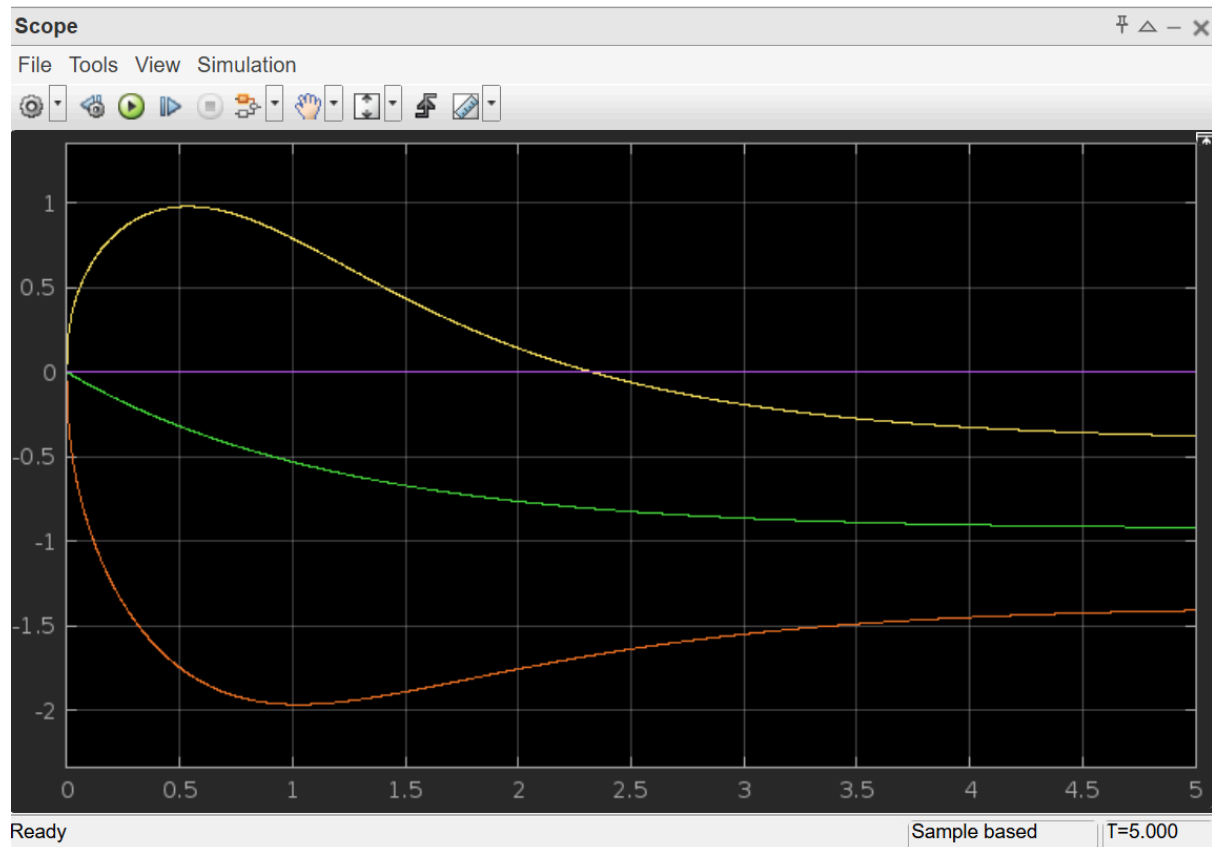
En la función `fcn2`, lo que hacemos es calcular únicamente la posición del extremo del robot. Reutilizamos la misma matriz de transformación homogénea, ya que contiene toda la información de la cinemática directa. De esa matriz, simplemente extraemos las coordenadas x , y y z , que representan la posición actual del TCP en el espacio.



En esta imagen se visualizan las velocidades articulares resultantes del control. Las curvas muestran valores iniciales elevados que decrecen con el tiempo, lo cual indica que el robot comienza el movimiento con mayor velocidad cuando el error espacial es grande y reduce su velocidad conforme se acerca al objetivo.



Vemos la evolución temporal de las coordenadas X, Y y Z del extremo del brazo robótico. Cada curva corresponde a uno de los ejes espaciales. Se puede observar cómo las posiciones convergen suavemente hacia valores constantes, lo cual demuestra que el manipulador efectivamente alcanza la posición objetivo deseada en el espacio. Esta convergencia suave y sin oscilaciones sugiere que el sistema es estable.



Finalmente, vemos la evolución de los ángulos articulares a lo largo del tiempo. Las curvas presentan transiciones desde sus posiciones iniciales hacia los valores que permiten alcanzar el objetivo en el espacio. El comportamiento de las curvas nuevamente muestra una evolución continua, sin oscilaciones pronunciadas ni inestabilidades, lo cual refuerza que el controlador implementado regula correctamente la postura del robot y lleva el TCP al objetivo de forma eficiente y controlada.